The Islamia University of Bahawalpur

Department of Information Technology

SOFTWARE REQUIREMENTS SPECIFICATION

(SRS document)

For

< VoteX: Secure Digital Voting Platform >

By

Inamil Noor

S23BINFT1M01066

Supervisor Dr. Muhammad Asad Ullah

S.No.	Chapter / Section	Page No.
1	**Introduction**	
1.1	Purpose	
1.2	Scope	
1.3	Definitions and Acronyms	
1.4	References	
2	**Overall Description**	
2.1	Product Perspective	
2.2	User Characteristics	
2.3	Operating Environment	
2.4	Constraints and Assumptions	
3	**Requirement Identifying Technique**	
3.1	Use Case Diagram	
3.2	Use Case Descriptions	
4	**Functional Requirements**	
4.1	User Authentication	
4.2	Voter Verification	

4.3	Election and Candidate Management	
4.4	Voting Process	
4.5	Encryption and Blockchain	
4.6	Tallying and Results	
4.7	Admin and Audit	
5	**Non-Functional Requirements**	
5.1	Security Requirements	
5.2	Performance Requirements	
5.3	Scalability and Reliability	
5.4	Usability and Accessibility	
5.5	Privacy and Compliance	
	References	

Revision History

Name	Date	Reason for Changes	Version
[Student Name]	[Date]	Initial draft of SRS document	1.0

Application Evaluation History

Comments (by Committee)	Action Taken
Include the ones given at scope time both in doc and presentation	

Supervised by [Supervisor Name]

Signature: _____

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to provide a comprehensive and detailed description of the requirements for VoteX: Secure Digital Voting Platform. This document serves as the foundational agreement between the development team, project supervisor, and all stakeholders regarding what the system shall do, how it shall behave, and the constraints under which it must operate.

VoteX is designed to modernize and secure the electoral process by transitioning from traditional paper-based voting to a robust, transparent, and tamper-proof digital voting system. The platform addresses critical challenges inherent in conventional voting systems, including voter fraud, ballot tampering, slow result compilation, lack of transparency, and logistical difficulties in managing large-scale elections.

This SRS document is intended to:

- Define the functional requirements that describe the specific behaviors, functions, and capabilities the VoteX platform must exhibit, including user registration, voter verification, ballot casting, vote encryption, result tallying, and administrative controls.
- Define the non-functional requirements that describe the quality attributes the system must possess, including security, performance, scalability, reliability, usability, accessibility, privacy, and compliance with legal and regulatory standards.
- Serve as a basis for system design and architecture, providing developers with clear guidelines on how the system should be structured, what technologies should be employed, and how various components should interact.
- Provide a reference for testing and validation, ensuring that every requirement stated in this document can be traced to specific test cases that verify correct implementation.
- Facilitate communication among stakeholders, including students, supervisors, evaluators, and potential end-users, by establishing a shared understanding of the system's capabilities and limitations.

The document follows the IEEE 830-1998 standard for Software Requirements Specifications and has been adapted to meet the academic requirements of the Final Year Project at the Department of Computer Science, The Islamia University of Bahawalpur.

It is important to note that VoteX is developed as an academic prototype. While every effort has been made to align the system with real-world security best practices and industry

standards, the platform is intended to demonstrate the feasibility and architecture of a secure digital voting system rather than to be deployed in an actual governmental or organizational election without further rigorous certification and auditing.

1.2 Scope

In-Scope:

- User registration with CNIC-based identity verification and multi-factor authentication (OTP).
- Election management (create, configure, schedule, activate, close).
- Candidate registration with photos, party affiliation, and manifestos.
- Digital ballot casting with review and confirmation steps.
- Vote encryption (AES-256) and blockchain-based immutable vote recording.
- Real-time automatic tallying and results dashboard with charts.
- Role-Based Access Control (RBAC) for Voter, Polling Officer, and Admin.
- Audit trail logging, email/SMS notifications, and PDF/CSV report generation.

Out-of-Scope:

- Government API integration (NADRA), biometric authentication, native mobile apps, offline voting, and legal election commission certification.

1.3 Definitions, Acronyms, and Abbreviations

TERM	DEFINITION
CNIC	Computerized National Identity Card
MFA	Multi-Factor Authentication
OTP	One-Time Password
RBAC	Role-Based Access Control
AES-256	Advanced Encryption Standard (256-bit)
SHA-256	Secure Hash Algorithm (256-bit)
JWT	JSON Web Token

1.4 References

- IEEE Std 830-1998 (SRS Standard)
- OWASP Top 10 (2021)
- Django & PostgreSQL Documentation

- Draw.io (Diagrams)

2. Overall Description

2.1 Product Perspective

VoteX exists in the emerging domain of digital democracy tools. Unlike simple survey tools, it integrates blockchain technology to ensure vote immutability. It is designed for the Pakistani context (CNIC verification) but operates independently without government API dependencies. It learns from systems like Estonia's i-Voting by adopting vote verification receipts while using blockchain to guarantee transparency.

2.2 User Characteristics

ACTOR	DESCRIPTION	TECHNICAL PROFICIENCY
Voter	Eligible individual casting vote	Low to High
Candidate	Nominated individual standing in election	Moderate
Polling Officer	Verifies voters and monitors elections	Moderate to High
Election Commissioner (Admin)	Manages entire system, elections, and users	High

2.3 Operating Environment

- Hardware: Standard web server (4-core CPU, 8GB RAM, 50GB SSD). Clients: any modern device.
- Software: Django (Python), PostgreSQL, Redis, Nginx, Gunicorn, Celery.
- Client Browsers: Chrome, Firefox, Safari, Edge (latest versions).
- Network: Stable internet required. HTTPS (TLS 1.3) enforced.

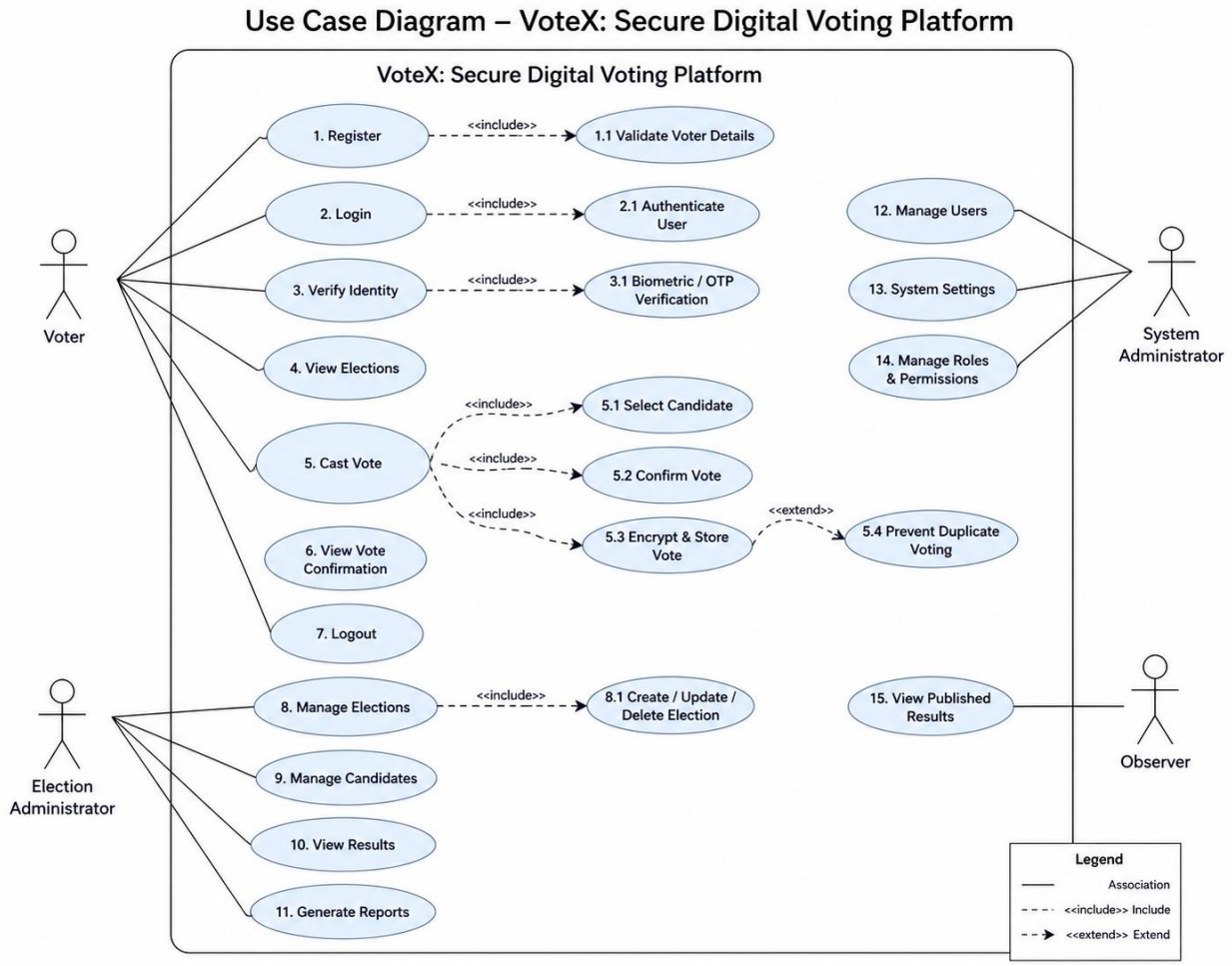
2.4 Constraints & Assumptions

- Constraints: Developed using Django/Python. No government API access. Academic timeline limits scope.

- Assumptions: Users have internet access and valid CNICs. Admins verify voters timely. A pre-existing voter registry is available.

3. Requirement Identifying Technique

3.1 Use Case Diagram



3.2 Use Case Descriptions

Table 1: Register

FIELD	DESCRIPTION
UC ID	UC-01
Actor	Voter

Description	User creates account with name, CNIC, email, mobile, and password.
Primary Flow	Enter details → Validate format → Check duplicates → Create account (status: Unverified) → Send OTP → Verify OTP.
Post-condition	Account created, pending verification.
Alternative Flow	Invalid CNIC/email format → Show error. Duplicate CNIC/email → Show error.

Table 2: Login

FIELD	DESCRIPTION
UC ID	UC-02
Actor	All Roles
Description	User authenticates via credentials + MFA (OTP).
Primary Flow	Enter email/CNIC + password → Validate → Send OTP → Verify OTP → Create JWT session → Redirect to dashboard.
Post-condition	User authenticated with active session.
Alternative Flow	Invalid credentials → Show error. 5 failed attempts → Lock account 30 mins.

Table 3: Upload CNIC & Verify

FIELD	DESCRIPTION
UC ID	UC-03
Actor	Voter, Polling Officer
Description	Voter uploads CNIC; Polling Officer approves/rejects against voter registry.
Primary Flow	Voter uploads JPG/PNG/PDF → Polling Officer views in queue → Cross-checks registry → Approves (status: Verified, generates Voter ID) or Rejects (with reason).
Post-condition	Voter status updated. Notification sent to voter.

Table 4: Create Election

FIELD	DESCRIPTION
UC ID	UC-04
Actor	Admin
Description	Admin creates election with title, type, positions, and schedule.
Primary Flow	Enter election details → Define positions → Set start/end time → Save as "Draft".
Post-condition	Election created in Draft status.
Alternative Flow	End time before start time → Show error.

Table 5: Register Candidate

FIELD	DESCRIPTION
UC ID	UC-05
Actor	Admin
Description	Admin registers candidate for a specific election position.
Primary Flow	Select election → Select position → Enter candidate details & photo → Save.
Post-condition	Candidate appears on the ballot.

Table 6: Cast Vote

FIELD	DESCRIPTION
UC ID	UC-06
Actor	Voter
Description	Verified voter selects candidates, reviews, and submits encrypted vote to blockchain.
Primary Flow	View ballot → Select one candidate per position → Review Ballot → Confirm → Encrypt (AES-256) → Hash (SHA-256) → Record on Blockchain → Mark as "Voted" → Show hash receipt.
Post-condition	Vote immutably recorded. Voter cannot vote again.
Alternative Flow	Already voted → Block. Missing selection → Prompt to select all.

Table 7: View Results

FIELD	DESCRIPTION
UC ID	UC-07
Actor	All Roles
Description	User views published election results with charts.
Primary Flow	Select completed election → View vote counts, percentages, winners → Download PDF.
Post-condition	Results viewed.

Table 8: Logout

FIELD	DESCRIPTION
UC ID	UC-08
Actor	All Roles
Description	User terminates session.
Primary Flow	Click Logout → Invalidate JWT → Redirect to Login.
Post-condition	Session terminated.

4. Functional Requirements

4.1 User Authentication

The system must allow users to register by providing their full name, CNIC number, date of birth, email address, mobile number, and a password. The password must be at least eight characters long and must include uppercase letters, lowercase letters, numbers, and special characters. The system must not allow two accounts with the same CNIC or the same email. After registration, the system must send a verification code to the user's email and phone. For logging in, the user must provide their email or CNIC along with their password, and then enter the verification code sent to them. If a user enters the wrong password five times in a row, the system must lock their account for thirty minutes and send them an email about it. The system must automatically log out any user who stays inactive for more than fifteen minutes. The system must also provide a password reset option through email with a link that expires in fifteen minutes.

4.2 Voter Verification

The system must allow registered users to upload a picture or scanned copy of their CNIC card. The file can be in JPG, PNG, or PDF format and must not be larger than 5 megabytes. The uploaded file must be saved in a secure folder that cannot be accessed through a web link. After uploading, the user must appear in a verification queue that is visible to the polling officer. The polling officer must be able to open the CNIC image, compare it with the user's details and the voter registry, and then either approve or reject the verification. If approved, the system must change the user status to verified and generate a unique voter ID. If rejected, the officer must provide a reason and the user must be notified by email. The system must allow a rejected user to upload their CNIC one more time.

4.3 Election and Candidate Management

The admin must be able to create an election by entering a title, description, type, start time, and end time. The admin must also be able to define positions within the election such as president, vice president, and secretary. The system must automatically change the election status from draft to active when the start time is reached, and from active to completed when the end time is reached. The admin must not be able to edit or delete an election once it is active. For candidates, the admin must be able to register a candidate by selecting an election and position, then entering the candidate name, uploading a photo,

adding a party name, and writing a manifesto. The system must shuffle the order of candidates on the ballot so that no candidate always appears first.

4.4 Voting Process

The system must show only those elections to a voter that are currently active and for which the voter is eligible. Only voters with verified status must be allowed to see the ballot. The ballot must clearly display all candidates for each position. The voter must select exactly one candidate for each position before proceeding. The system must show a review page where the voter can see all their selected candidates before the final submission. The voter must explicitly click a confirm button to cast the vote. The system must strictly prevent any voter from casting more than one vote in the same election.

4.5 Encryption and Blockchain

The system must encrypt every vote using the AES-256 method before saving it. This means that even if someone gains access to the database, they cannot read the votes without the secret key. The system must generate a SHA-256 hash for each encrypted vote which acts as a unique fingerprint. The system must then save this encrypted vote as a block on a private blockchain. Each block must contain the vote data, a timestamp, the hash of the previous block, and its own hash. This creates a chain where changing one block would break all the blocks after it, making the data tamper-proof. The system must keep the voter identity completely separate from the vote data so that nobody can find out who voted for whom. After voting, the system must give the voter a receipt containing their hash code so they can verify their vote was recorded.

4.6 Tallying and Results

When an election ends, the system must automatically count all the votes by decrypting them and adding them up. The system must calculate the total votes and percentage for each candidate for every position. The candidate with the most votes must be declared the winner. However, the results must not be shown to voters until the admin reviews them and clicks a publish button. Once published, the results must be displayed on a public dashboard using bar charts and pie charts so that they are easy to understand. The system must also allow users to download the results as a PDF or CSV file.

4.7 Admin and Audit

The system must have a Role-Based Access Control system where each user role has specific permissions. The voter has the lowest access, the polling officer has moderate access to verification tools, and the admin has full access to everything. The system must keep a detailed log of every important action that happens in the system. This includes every login attempt, every vote that is cast, every change made by an admin, and every error that occurs. Each log entry must include the time, the user involved, their IP address, and a description of what happened. The admin must be able to filter these logs by date and type, and export them as a CSV file. The system must also send automatic email and SMS notifications for important events like registration success, verification status change, and election announcements.

5. Non-Functional Requirements

5.1 Security Requirements

Security is the most important requirement for a voting system. The system must use HTTPS for all communication so that no one can spy on the data traveling between the user and the server. The system must protect itself from common web attacks like SQL injection, cross-site scripting, and cross-site request forgery using the built-in protections provided by the Django framework. All passwords must be stored in a hashed form using the bcrypt method, meaning the original password is never saved anywhere. The encryption key used to lock the votes must be stored in an environment variable and never in the code or database. The system must limit how many times someone can try to log in from the same address to prevent automated attacks. All cookies used by the system must have security flags that prevent them from being stolen by scripts.

5.2 Performance Requirements

The system must be fast enough to provide a smooth experience. Any page in the system must load within three seconds under normal conditions. The login process must complete within two seconds. The most important performance requirement is that the entire vote casting process including encryption and blockchain recording must finish within five seconds so the voter does not have to wait. The system must be able to handle at least five hundred users using the system at the same time without slowing down. Database queries must run within half a second even when there are large amounts of data.

5.3 Scalability and Reliability

The system must be designed in a way that allows it to grow. If more users start using the system, it should be possible to add more server instances to handle the load. Redis must be used for storing sessions and caching data so that multiple servers can share session information. The system must use Celery to handle slow tasks like sending emails and SMS in the background so that the user does not have to wait for them. The system must aim to be available at least ninety-nine and a half percent of the time during an active election. The database must be backed up automatically every day and the backups must be kept for at least ninety days.

5.4 Usability and Accessibility

The system must have a clean and professional design that is easy to understand for people with different levels of computer skills. The voting process from opening the ballot to confirming the vote must be completable in fewer than ten clicks. All error messages must be written in simple language that tells the user exactly what went wrong and how to fix it. The system must show loading signs for any action that takes more than one second. The system must ask for confirmation before any destructive action like deleting an election or rejecting a voter. The system must also support basic accessibility features like keyboard navigation and proper color contrast so that it can be used by people with disabilities.

5.5 Privacy and Compliance

The system must guarantee complete ballot secrecy. This means that no person, not even the admin or the developer, should be able to find out which candidate a specific voter chose. The system must encrypt all personal information like CNIC and date of birth when storing it in the database. The system must not share any user data with third parties. The system must display a clear privacy policy and terms and conditions page that users must agree to before registering. Most importantly, the system must display a visible disclaimer stating that VoteX is an academic project and is not certified or approved by any election commission for real-world use.

References

1. IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.

2. OWASP Foundation, OWASP Top Ten Web Application Security Risks, 2021.
3. Django Software Foundation, Django Documentation, docs.djangoproject.com.
4. PostgreSQL Global Development Group, PostgreSQL Documentation, postgresql.org/docs.
5. Redis Inc., Redis Documentation, redis.io/docs.
6. Draw.io, Online Diagram Software, app.diagrams.net.